BUSINESS BLUEPRINT

Migración del Modelo de Pronóstico de Ventas de Databricks a Snowflake y Evaluación de Capacidades MLOps End-to-End

Versión 3.0 · Febrero 2026 · Confidencial

BUSINESS BLUEPRINT — APROBACIÓN			
Documento	**Nombre**	**Fecha de aprobación**	**Firma**

Guía de Lectura del Documento

Este documento está organizado en tres bloques estratégicos claramente diferenciados. Cada sección está etiquetada con un indicador de alcance para facilitar la comprensión del estado actual frente a la evolución futura:

[POC]	Implementado y validado en la Prueba de Concepto. Componentes reales, ejecutables y demostrados.
[PROD. CERCANO]	Mejoras y componentes por formalizar en la siguiente fase hacia ambiente productivo.
[PROD. LEJANO]	Roadmap estratégico: automatización avanzada, escalabilidad y observabilidad ampliada.

1. Introducción

Arca Continental manifestó interés en evaluar de extremo a extremo las capacidades de Snowflake para desplegar modelos de ML mediante MLOps (Model Registry, Feature Store, ML Observability) y abordar la migración de su modelo de Pronóstico de Ventas Semanales, actualmente en ejecución en Databricks, hacia Snowflake.

Este documento describe la arquitectura técnica propuesta, el alcance de la Prueba de Concepto (POC) realizada y la evolución planificada hacia un ambiente productivo robusto, gobernado y escalable.

2. Objetivos

El objetivo principal es plantear y validar la migración del modelo de ML de Pronóstico de Ventas Semanales (uni_box_week), el cual fue analizado mediante el Snowpark Migration Accelerator (SMA), hacia Snowflake, aprovechando la oportunidad para evaluar el ciclo de vida completo de MLOps con las mejores prácticas de la plataforma.

Los objetivos específicos incluyen:

- Validar la factibilidad técnica del ciclo completo MLOps en Snowflake (entrenamiento, registro, inferencia, monitoreo).
- Reproducir funcionalmente la arquitectura MLOps actualmente implementada en Databricks.
- Simplificar la operación mediante capacidades nativas de Snowflake ML, reduciendo la complejidad y la dependencia de componentes externos.
- Establecer las bases técnicas para la evolución hacia un ambiente productivo automatizado, gobernado y auditable.

3. Alcance

3.1 Estándar Arquitectónico Adoptado

La arquitectura propuesta homologa el uso de tablas físicas (tablas fijas) como estándar de almacenamiento en lugar de Dynamic Tables o Snowflake Tasks. Esta decisión responde a las restricciones de permisos actuales del entorno de desarrollo, y se mantendrá también en la estrategia de orquestación productiva.

Decisiones de Estandarización

- Tablas físicas (fijas) como estándar de almacenamiento para features y resultados.
- Cálculo de lags y ventanas temporales mediante SQL, integrados en dichas tablas.
- Python (Snowpark) y SQL como únicos lenguajes en los notebooks.
- Sin uso de Snowflake Tasks ni tablas dinámicas en la POC por restricciones de permisos.
- Orquestación externa o manual hasta definición formal de la estrategia productiva.

El uso exclusivo de Python y SQL responde a los siguientes criterios técnicos:

- Compatibilidad nativa con las APIs de Snowflake ML (snowflake.ml.modeling, Feature Store, Registry).
- Estandarización del stack tecnológico, facilitando el mantenimiento y la transferencia de conocimiento.
- Gobernanza y auditabilidad: los notebooks en Python + SQL son inspeccionables, versionables y reproducibles dentro del ecosistema Snowflake.
- Eliminación de dependencias externas (frameworks adicionales, entornos virtuales complejos, librerías de orquestación de terceros).

3.2 Lo que Incluye la POC

▶ POC

- Ejecución del entrenamiento de modelos sobre datasets previamente preparados y disponibles en Snowflake.
- Implementación del Feature Store como tablas físicas con features calculadas mediante SQL (lags, rolling aggregations).
- Búsqueda de hiperparámetros (HPO) por grupo utilizando Bayesian Search (snowflake.ml.modeling.tune).
- Entrenamiento de 16 sub-modelos en paralelo mediante Many Model Training (MMT).
- Registro y versionado de modelos en el Snowflake Model Registry.
- Inferencia batch particionada mediante sintaxis SQL nativa.
- Configuración inicial de ML Observability para monitoreo de desempeño y drift.

3.3 Lo que NO Incluye la POC

- Ingesta, limpieza o transformación de datos desde fuentes origen (los datasets son provistos ya curados por Arca Continental).
- Diseño o implementación de pipelines de datos bajo el modelo Medallón.
- Definición o ajuste de métricas de negocio, KPIs o umbrales finales de aceptación.
- Orquestación completamente automatizada end-to-end (Snowflake Tasks, ML Jobs).
- Parametrización dinámica mediante tablas de configuración centralizadas.
- CI/CD automatizado con validación en cada etapa.
- Real-time inference y endpoints de serving (SPCS).

4. Datasets

Los datasets utilizados en la POC fueron provistos por Arca Continental de forma curada y lista para su uso como fuente de entrenamiento e inferencia:

Dataset	Volumen	Descripción
Training	~96 millones de filas (2.9 GB)	Ventas semanales por cliente. Información del año 2025 dividida por semanas. Segmentado en 16 grupos (STATS_NTILE_GROUP).
Inference	~24.7 millones de filas (807.2 MB)	Misma granularidad que el dataset de entrenamiento. Utilizado para validar predicciones y evaluar el comportamiento productivo del modelo.

Gestión del Test Set y Predicciones Productivas

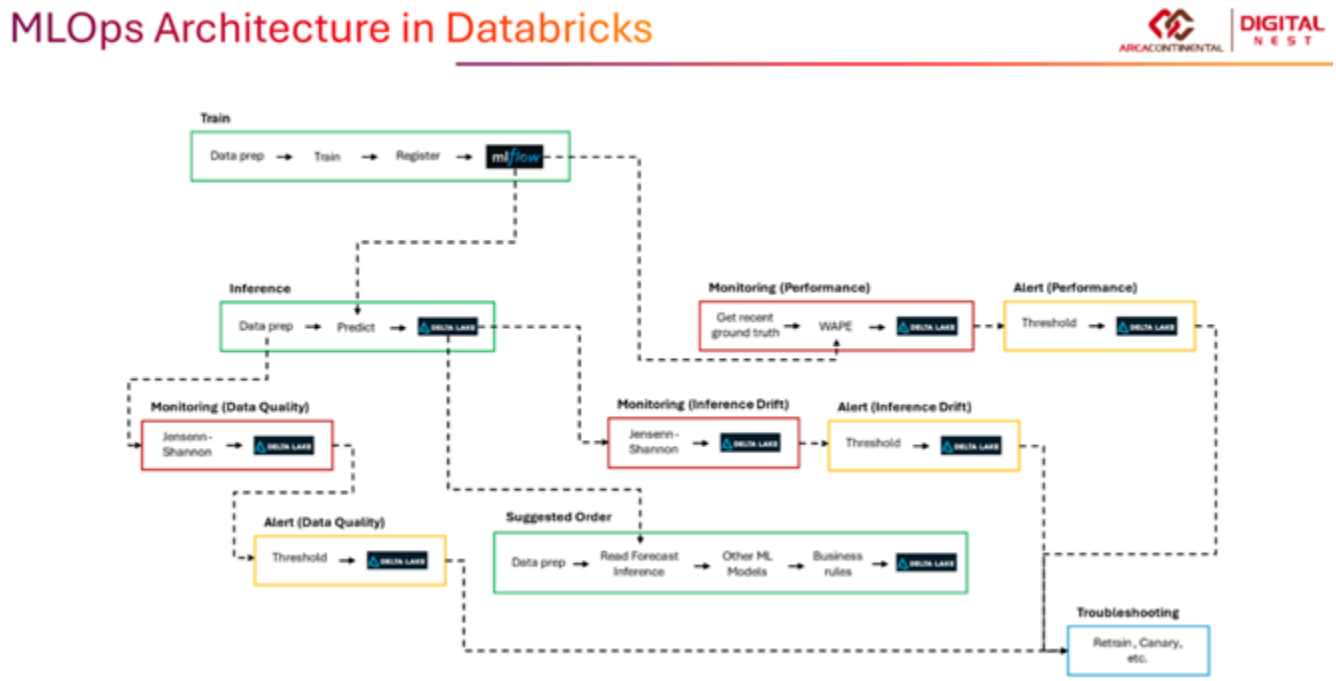
▶ POC

Una consideración crítica de diseño es la diferenciación entre el conjunto de prueba (test set) utilizado durante el entrenamiento y las predicciones productivas semanales. Estas dos operaciones sirven propósitos distintos y deben mantenerse separadas arquitectónicamente:

- **Test Set:** Subconjunto del dataset de entrenamiento reservado para evaluación de métricas. Se utiliza un split 80/20, donde el 80% se destina a entrenamiento y el 20% a prueba (validación). Esta proporción ofrece un balance práctico entre suficiente volumen de datos para que el modelo aprenda patrones robustos y un tamaño de muestra adecuado para obtener métricas estadísticamente estables y representativas. El split se realiza de forma estratificada por grupo (STATS_NTILE_GROUP) para mantener la distribución en cada segmento.
- **Predicciones productivas:** Se ejecutan únicamente sobre el dataset de inferencia correspondiente a la semana en curso, sin mezcla con el test set de entrenamiento.

5. Arquitectura Actual en Databricks

El siguiente diagrama representa la arquitectura MLOps en Databricks, compartida por Arca Continental como referencia para el análisis y definición de la arquitectura objetivo en Snowflake.



La arquitectura actual implementa un enfoque de entrenamiento de múltiples modelos especializados. Los puntos clave son:

1. Segmentación del dataset de entrenamiento en 16 grupos basados en patrones recientes de venta (últimas 4 semanas) y volumen histórico por cuartiles (0-25%, 25-50%, 50-75%, 75-100%).
2. Entrenamiento distribuido manual: 16 sub-modelos entrenados individualmente con AutoML de Databricks (60 min timeout por modelo), resultando en ~16 horas secuenciales de cómputo.
3. Ensemble Orchestration Custom: librería "Pandas Ensemble" personalizada que unifica los 16 sub-modelos en un objeto lógico único con ruteo automático por grupo en inferencia.
4. Gestión con MLflow: versionado por timestamp, aliases por fecha de ejecución (ej. 20250910), tags con execution_date y cutoff_date.
5. Monitoreo manual con alertas no proactivas: Data Drift (Jensen-Shannon >0.2 Warning, >0.4 Critical), Prediction Drift por distribución histórica vs actual, y Performance Drift con WAPE + F1 Score retroactivo.

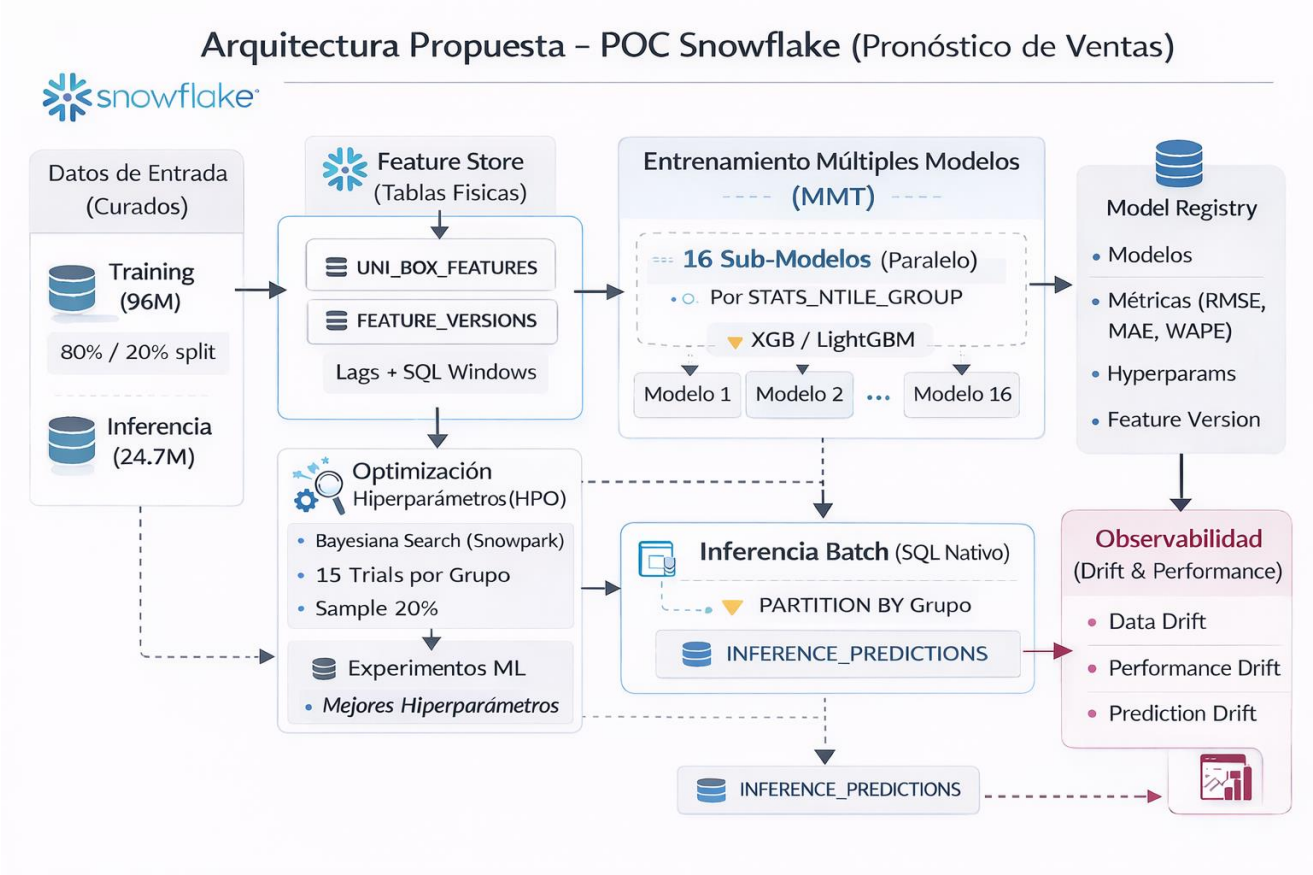
6. Arquitectura Propuesta en Snowflake

6.1 Visión General de la POC

▶ POC

La arquitectura propuesta para la POC en Snowflake tiene como objetivo reproducir el flujo MLOps de Databricks manteniendo la equivalencia funcional del modelo de Pronóstico de Ventas, pero simplificando su operación mediante capacidades nativas de Snowflake ML.

Procesos como el entrenamiento de múltiples sub-modelos, el ruteo de inferencias, el versionado de modelos y el monitoreo del desempeño se resuelven mediante HPO manual + MMT, ML Experiments y ML Registry, en lugar de AutoML propietario y componentes custom.



6.2 Comparativa de Capacidades MLOps

La siguiente tabla resume la equivalencia funcional entre Databricks y Snowflake para cada componente del ciclo MLOps:

Componente	Databricks (actual)	Snowflake (propuesto)
Entrenamiento	AutoML Databricks + loops manuales de segmentación	HPO Bayesian Search + Many Model Training (MMT), como una alternativa parcial. Revisar sección 8.3.
Inferencia	Pandas Ensemble custom (PyFunc) con ruteo manual	Partitioned Model Inference con ruteo automático nativo. Revisar sección 10.

Gestión de Modelos	MLflow externo con aliases y tags	Snowflake Model Registry integrado (versiones, tags). Revisar sección 12.
Orquestación	Azure Data Factory	POC: manual. Revisar sección 13.
Observabilidad	Notebooks custom de monitoreo desacoplados	ML Observability con monitores custom. Revisar sección 11.
Feature Management	Tablas Delta en Databricks	Tablas físicas + SQL (lags calculados, sin FeatureView por permisos). Revisar sección 7.
Seguridad	Permisos a nivel de catálogo Databricks	RBAC nativo Snowflake (roles ML_DEV, ML_OPS, ML_CONSUMER). Revisar sección 12.4.

7. Feature Store

7.1 Implementación en la POC — Tablas Físicas con Cálculo Manual

▶ POC

En la POC, el Feature Store se implementa mediante tablas físicas (tablas planas fijas) en el schema SC_FEATURES_BMX. Esta decisión responde a restricciones de permisos del entorno de desarrollo que impiden el uso de Dynamic Tables o Snowflake Tasks para actualizaciones automáticas.

El flujo de construcción de features es el siguiente:

1. La tabla BD_AA_DEV.SC_STORAGE_BMX_PS.TRAIN_DATASET_STRUCTURED se toman como fuente única.
2. Se ejecutan transformaciones SQL directamente en Snowflake para re-calcular features de lag (ventas de las últimas N semanas), rolling aggregations (sumas, promedios, máximos en ventanas de 4, 12 y 24 semanas) y variables de comportamiento por cliente-producto.
3. Los resultados se materializan en la tabla UNI_BOX_FEATURES dentro del schema de features, lista para ser consumida tanto por el proceso de entrenamiento como por inferencia.

Las features disponibles en la POC incluyen ventanas temporales de ventas pasadas (SUM_PAST_4_WEEKS, AVG_PAST_12_WEEKS, MAX_PAST_24_WEEKS, entre otras), métricas de comportamiento por punto de venta (NUM_COOLERS, NUM_DOORS) y variables categóricas de segmento (PHARM_SUPER_CONV, WINES_LIQUOR, GROCERIES, SPEC_FOODS).

Cálculo de Lags mediante SQL

Los lags y ventanas temporales se calculan con sentencias SQL estándar utilizando funciones de ventana (WINDOW FUNCTIONS). Por ejemplo:

SUM(ventas) OVER (PARTITION BY cliente, producto ORDER BY semana ROWS BETWEEN 12 PRECEDING AND 1 PRECEDING)

Este enfoque es reproducible, auditable y compatible con el engine de Snowflake sin dependencias adicionales.

7.2 Evolución del Feature Store

▶ PROD. CERCANO

En producción cercano, el Feature Store evoluciona hacia una arquitectura batch incremental con trazabilidad completa por lote. Esta arquitectura responde a los requerimientos de Arca Continental: trazabilidad por batch procesado, capacidad de auditoría y re-ejecución de lotes específicos, y compatibilidad con el esquema de cortes que ya manejan actualmente.

Componentes de la Arquitectura

Tabla	Schema	Descripción
TRAIN_TRANSACTIONS	SC_FEATURES_BMX	Tabla fuente con columnas técnicas de control: BATCH_ID, PROCESS_STATUS (PENDING / IN_PROGRESS / PROCESSED / FAILED), PROCESSED_AT. Define el corte de cada ejecución.
FEATURE_BATCH_RUNS	SC_FEATURES_BMX	Control de auditoría por lote: BATCH_ID, RUN_TS_START, RUN_TS_END, STATUS,

		conteos de registros procesados y notas de la ejecución.
FEATURES_UNI_BOX_BATCH	SC_FEATURES_BMX	Staging del batch actual: features calculadas en esta ejecución (ENTITY_ID, FEATURE_TS, BATCH_ID + columnas de features). Se sobreescribe en cada corrida.
FEATURES_UNI_BOX_HIST	SC_FEATURES_BMX	Histórico append-only: acumula todas las ejecuciones con su BATCH_ID. Fuente para entrenamiento: permite reproducir el estado de features de cualquier batch pasado.
FEATURES_UNI_BOX_LATEST	SC_FEATURES_BMX	Estado actual (DROP/RECREATE): contiene las entidades activas del último batch. Fuente optimizada para inferencia. No mantiene entidades inactivas de ejecuciones anteriores.

Decisión clave: DROP/RECREATE en FEATURES_UNI_BOX_LATEST (no MERGE)

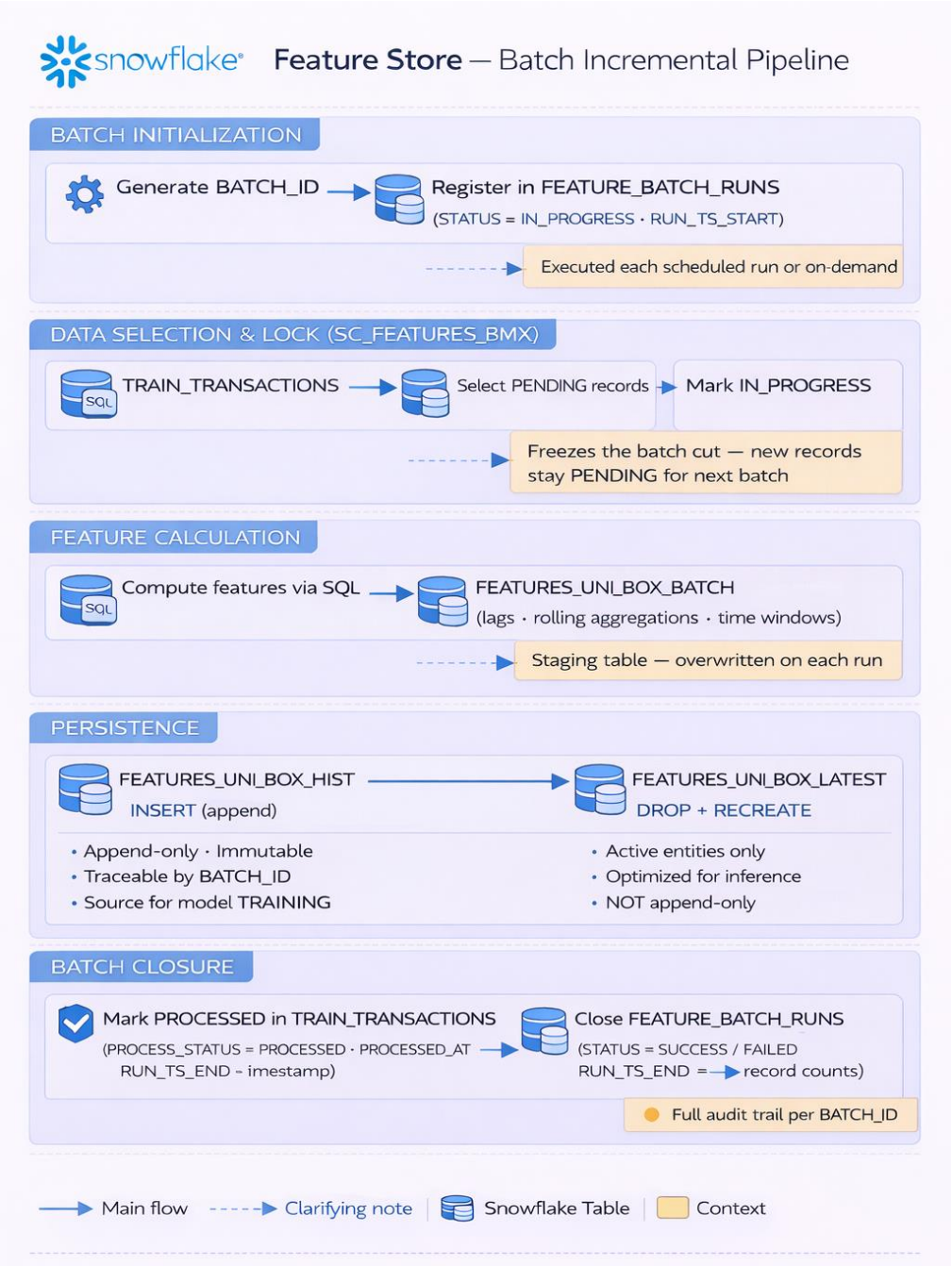
La tabla LATEST no se actualiza mediante MERGE/UPSERT por ENTITY_ID. En cada ejecución de batch, la tabla se elimina y se recrea con el contenido del batch actual, esto siempre y cuando el batch el proceso sea parte del grano temporal más reciente (semana más alta, mes más alto, etc). Esto significa que LATEST refleja exactamente las entidades transaccionalmente activas en el último lote, sin arrastrar entidades inactivas de ejecuciones anteriores. El histórico completo siempre está disponible en FEATURES_UNI_BOX_HIST (append-only), donde cada registro mantiene su BATCH_ID para trazabilidad. Para añadir nuevas features: se necesita cambiar el proceso de compute features para los siguientes batches, para la estructura histórica se aplica ALTER TABLE para agregar columnas (en una primera instancia de prueba este proceso seria de forma manual, posteriormente se pude tener una tabla de configuración la cual verifique las features existentes y a partir de esto tener un alter automatizado). Las features no se eliminan a nivel estructural — solo se excluyen del cómputo si ya no se necesitan, preservando así la trazabilidad del histórico.

Flujo del Pipeline Batch

1. Generación de BATCH_ID e inicio del registro en FEATURE_BATCH_RUNS (STATUS = IN_PROGRESS, RUN_TS_START).
2. Selección de transacciones con PROCESS_STATUS = PENDING desde TRAIN_TRANSACTIONS.
3. Marcado IN_PROGRESS con el BATCH_ID actual: congela el corte. Nuevas transacciones que lleguen en paralelo quedan en PENDING para el siguiente batch.
4. Cálculo de features para el batch actual (lags, rolling aggregations, ventanas temporales) mediante SQL. Resultado en FEATURES_UNI_BOX_BATCH.
5. INSERT en FEATURES_UNI_BOX_HIST (append). El histórico es inmutable y trazable por BATCH_ID.
6. DROP y RECREATE de FEATURES_UNI_BOX_LATEST con el contenido del batch actual. La tabla refleja solo las entidades activas del lote.
7. Marcado de transacciones como PROCESSED en TRAIN_TRANSACTIONS.
8. Cierre del batch en FEATURE_BATCH_RUNS: STATUS = SUCCESS o FAILED, RUN_TS_END y conteos finales.

Uso por Componente del Pipeline

- Entrenamiento (HPO + MMT): consume FEATURES_UNI_BOX_HIST filtrando por BATCH_ID o rango temporal. Permite reproducir exactamente el estado de features de cualquier ejecución pasada.
- Inferencia batch: consume FEATURES_UNI_BOX_LATEST directamente. Al ser DROP/RECREATE, el contenido es siempre el del último batch activo sin filtros adicionales.



Orquestación del Feature Store

▶ PROD. CERCANO

- Opción A — Snowflake Tasks (si se habilitan permisos): pipeline nativo con dependencias y logging integrado. Actualmente no disponible en la POC por restricciones de permisos.
- Opción B — Snowflake ML Jobs: alternativa orientada a cargas ML. Requiere permisos similares a Tasks pero puede habilitarse de forma independiente.

- Opción C — Orquestador externo (Dagster u otro): el pipeline se implementa en notebooks orquestados desde Dagster, invocando las operaciones SQL en Snowflake vía conector. No requiere permisos adicionales en Snowflake.

Alcance de la POC (Ultima estructura)

▶ POC

- Creación de tabla de logs para batch runs (Trazabilidad del batch).
- Alter de tabla de transacciones para identificar el proceso/status del registro y batch que representa
- Modificación del proceso para compute features (Agregar inserción de batch alineado a la tabla de batch runs, modificación de salida de batch hacia FEATURES_UNI_BOX_BATCH).
- El proceso de compute features/FEATURES_UNI_BOX_BATCH, solamente trabajar con aquellas transacciones las cuales tienen un estado de pendiente o nulo.
- Una vez terminado el módulo de compute features el estado de las transacciones pueden adquirir el valor de nulo/pendiente/procesado.
- Compute Features (FEATURES_UNI_BOX_BATCH) NO es dinámico (No estará basado en una tabla de configuración para creación de features basados en sqls statments).
- El módulo de compute features es estático en estructura, si se desea agregar features se tiene que crear en el módulo de staging necesario para su creación dependiendo de su granularidad y dependencia, así mismo el proceso de joins basado en la entidad de las features, para la creación final de la tabla FEATURES_UNI_BOX_BATCH.
- Después del que el proceso de FEATURES_UNI_BOX_BATCH finalice el contenido puede tener 2 caminos de escritura; sobre la tabla de historia y Latest agregando los datos técnicos de trazabilidad (batch_id), esta inserción es mediante un query estático.
- El batch procesado asumiendo siempre que tenga una estructura correcta debe de ser trasladado a la tabla de historia junto con su batch_id.
- El batch que fue procesado en FEATURES_UNI_BOX_BATCH solamente pasaría a la tabla de Latest si es que el conjunto temporal es igual o mayor del presente en la tabla Latest.
- La tabla FEATURES_UNI_BOX_LATEST se recrea una vez que la entidad temporal de FEATURES_UNI_BOX_BATCH iguala o supera la entidad temporal actual de la misma tabla Latest.
- No se tiene un proceso de QA de datos de entrada.
- No se tiene un proceso de QA dentro de la inserción de cualquier parte de las tablas creadas para feature engineering.

8. Entrenamiento: HPO y Many Model Training

8.1 HPO — Hyperparameter Optimization

▶ POC

Qué es y qué resuelve

HPO (Hyperparameter Optimization) es el proceso de búsqueda sistemática de la combinación óptima de hiperparámetros para cada algoritmo y segmento. HPO automatiza la exploración y selecciona la configuración que minimiza el error de validación.

En la POC, HPO también puede considerarse una forma de orquestación manual, ya que es el componente que controla qué algoritmo y qué configuración se asigna a cada uno de los 16 grupos, registrando los resultados de forma trazable en ML Experiments.

Implementación y decisiones técnicas

El proceso de HPO sigue el siguiente flujo para cada uno de los 16 grupos de segmentación:

1. Se carga el dataset del grupo desde la tabla de features (UNI_BOX_FEATURES) o directamente desde la tabla de entrenamiento limpia como fallback.

2. Se ejecutan **15 trials** de Bayesian Search por grupo, valor que equilibra calidad de exploración y tiempo de cómputo (≈1 hora para los 16 grupos sobre el cluster de 5 nodos M). Este número es configurable: más trials amplían el espacio explorado a costa de mayor tiempo; menos trials aceleran el proceso pero pueden derivar en soluciones subóptimas.
3. Se aplica un muestreo estratificado del 20% del dataset del grupo para la fase de búsqueda. Aunque en primera instancia pueda parecer una decisión arbitraria, tiene una justificación técnica sólida: con ~96M de filas (2.9 GB) distribuidos en 16 grupos, entrenar 15 trials con el 100% de los datos implicaría tiempos de exploración de muchas horas por grupo. El 20% mantiene suficiente representatividad estadística para discriminar configuraciones buenas de malas, que es el objetivo del HPO, sin incurrir en el costo de entrenamiento completo. El entrenamiento final con MMT sí utiliza el 100% de los datos.
4. Se realiza un split 80/20 dentro del subconjunto muestreado para train/validation, estratificando por la distribución del target.
5. Se ejecuta Bayesian Search (BayesOpt) con 15 trials y hasta 4 trials concurrentes por grupo. El proceso está estructurado en un bucle secuencial a nivel lógico (un grupo tras otro), pero internamente cada trial de HPO se ejecuta en paralelo sobre el cluster de 5 nodos M, al igual que MMT. Esto reduce significativamente el tiempo real de exploración respecto a una ejecución puramente secuencial.
6. Los hiperparámetros optimizados incluyen: número de estimadores, profundidad máxima, tasa de aprendizaje, parámetros de regularización (reg_alpha, reg_lambda), subsampling y colsample_bytree.
7. Los resultados (mejores parámetros, RMSE, MAE, WAPE por grupo) se registran en ML Experiments para trazabilidad, con fallback a tabla HYPERPARAMETER_RESULTS si ML Experiments no está disponible.

¿Por qué Bayesian Search y no Random Search o Grid Search?

Grid Search evalúa todas las combinaciones posibles del espacio de hiperparámetros: con 9 parámetros y rangos continuos, el número de combinaciones es computacionalmente prohibitivo.

Random Search muestrea combinaciones al azar de forma ciega, sin aprender de las evaluaciones anteriores. Es mejor que Grid, pero ineficiente en espacios de alta dimensión.

Bayesian Search (BayesOpt) construye un modelo probabilístico del espacio de hiperparámetros a partir de los trials previos, y dirige los siguientes trials hacia las regiones con mayor probabilidad de mejorar la métrica objetivo. Esto permite encontrar configuraciones óptimas con un número mucho menor de evaluaciones, lo que se traduce directamente en menor tiempo de cómputo y menor costo de warehouse.

El mapeo de algoritmo por grupo es el siguiente (resultado del análisis de la POC, el cual consistió en validar la compatibilidad entre las librerías nativas de modelos ML en Snowflake vs las usadas en Databricks):

```
GROUP_MODEL = {
  "group_stat_0_1": "LGBMRegressor",
  "group_stat_0_2": "LGBMRegressor",
  "group_stat_0_3": "LGBMRegressor",
  "group_stat_0_4": "LGBMRegressor",
  "group_stat_1_1": "LGBMRegressor",
  "group_stat_1_2": "LGBMRegressor",
  "group_stat_1_3": "XGBRegressor",
  "group_stat_1_4": "XGBRegressor",
  "group_stat_2_1": "LGBMRegressor",
  "group_stat_2_2": "LGBMRegressor",
  "group_stat_2_3": "XGBRegressor",
  "group_stat_2_4": "XGBRegressor",
  "group_stat_3_1": "LGBMRegressor",
  "group_stat_3_2": "LGBMRegressor",
  "group_stat_3_3": "LGBMRegressor",
  "group_stat_3_4": "XGBRegressor",
}
```

En la POC, el algoritmo asignado a cada grupo está definido explícitamente en el código, tomando como referencia los modelos ya utilizados por Arca Continental en Databricks. Esta decisión reduce el riesgo de la migración al mantener una base conocida y validada por el negocio. A futuro, la sección 8.3 describe cómo automatizar esta selección mediante una capa de orquestación sobre HPO, eliminando la necesidad de definir el algoritmo manualmente por segmento.

Frecuencia de ejecución

- **En la POC:** HPO se ejecuta manualmente una vez antes del primer entrenamiento completo.
- **En producción cercana:** HPO se ejecutará de forma puntual bajo demanda, cuando el equipo ML detecte degradación de performance o un cambio significativo en la distribución de los datos.
- **En producción lejana:** HPO se ejecutará de forma automática, disparado por los monitores de ML Observability ante drift significativo, evaluando además múltiples algoritmos candidatos por segmento según la propuesta de AutoML descrita en la sección 8.3.

8.2 MMT — Many Model Training

▶ POC

Qué es y qué resuelve

Many Model Training (MMT) es una funcionalidad nativa de Snowflake que permite entrenar múltiples modelos de forma paralela, donde cada modelo se especializa en un subconjunto específico de los datos definido por una columna de partición. En el contexto de ARCA, esta columna es STATS_NTILE_GROUP, que define los 16 segmentos del modelo.

Mientras que HPO optimiza la configuración para cada grupo, MMT entrena los 16 modelos finales de forma simultánea usando esos parámetros validados. Esto simplifica radicalmente la orquestación: en Databricks se requieren loops manuales, diccionarios de tracking (experiment_submodel_map, group_submodel_map) y gestión manual de artefactos; MMT automatiza todo con una sola llamada especificando la columna de partición.

Implementación y tiempos de ejecución

El proceso de MMT sigue el siguiente flujo:

1. Se cargan los hiperparámetros óptimos por grupo desde ML Experiments o desde la tabla de resultados de HPO.
2. Se carga el dataset completo (100% de los datos, ~96M filas / 2.9 GB) desde la tabla de features.
3. Se define la función de entrenamiento por segmento, que recibe los hiperparámetros específicos del grupo e instancia el modelo correspondiente (XGBRegressor o LGBMRegressor según el mapeo por grupo).
4. Se ejecuta ManyModelTraining con particionamiento por STATS_NTILE_GROUP. Snowflake distribuye automáticamente los 16 grupos entre los nodos del cluster, ejecutándolos en paralelo sin intervención manual.
5. El entrenamiento completo de los 16 sub-modelos sobre el 100% del dataset (~96M filas / 2.9 GB) tarda aproximadamente 7 minutos en el cluster de 5 nodos M.
6. Cada modelo se registra en el Snowflake Model Registry con su versión, métricas (RMSE, MAE, WAPE) y timestamp de entrenamiento.

```
# Fragmento ilustrativo del flujo MMT
from snowflake.ml.modeling.distributors.many_model import ManyModelTraining

mmt = ManyModelTraining(
    session=session,
```

```
    partition_column="STATS_NTILE_GROUP",
    train_func=train_func_with_hpo_params,  # función que carga params por grupo
)
mmt.run(train_df)  # Snowflake paraleliza automáticamente los 16 grupos
```

Frecuencia de ejecución

- **En la POC:** MMT se ejecuta manualmente una vez por ciclo de experimentación, bajo demanda del equipo ML.
- **En producción cercana:** MMT se ejecutará de forma periódica (frecuencia semanal o mensual por definir con Arca Continental), disparado manualmente o mediante un orquestador externo al inicio de cada ciclo de negocio.
- **En producción lejana:** MMT se ejecutará de forma automática, disparado por trigger de drift detectado por ML Observability o por calendario, con validación de métricas integrada antes de promover el modelo resultante a producción.

Escalabilidad de Modelos

La estrategia de escalamiento adoptada es la segmentación por columna de partición (STATS_NTILE_GROUP). Este enfoque fue seleccionado sobre alternativas como partición lógica por región o distribución computacional genérica, porque:

- Preserva la lógica de negocio existente en Databricks (16 grupos basados en patrones de venta y volumen).
- Alinea el escalamiento con el patrón de MMT de Snowflake, que está optimizado para la paralelización por partición.
- Permite asignar algoritmos distintos por segmento (XGBoost para grupos de alto volumen, LightGBM para grupos de baja densidad) sin modificar la arquitectura base.
- El código escala automáticamente sin modificaciones: al aumentar el número de grupos o el tamaño del warehouse, MMT distribuye la carga entre nodos disponibles.

8.3 Propuesta Conceptual: AutoML en Snowflake

▶ **PROD. LEJANO**

Contexto: ¿por qué los modelos están fijos en el código hoy?

En la POC actual, el algoritmo asignado a cada grupo (XGBoost o LightGBM) está definido explícitamente en el código de entrenamiento. Esto significa que para cambiar el modelo de un segmento hay que modificar el código manualmente.

Arca Continental señaló la necesidad de automatizar esta selección, de modo que el sistema pueda evaluar múltiples algoritmos por segmento y elegir el óptimo sin intervención humana.

La propuesta conceptual que se describe a continuación resuelve exactamente esta necesidad, elevando el HPO actual a un AutoML completo dentro del ecosistema Snowflake.

Snowflake no dispone actualmente de una solución AutoML nativa equivalente a la de Databricks. No obstante, es posible construir un enfoque AutoML utilizando las capacidades existentes de Hyperparameter Optimization (HPO) y Many Model Training (MMT).

En la POC actual, el proceso de HPO optimiza hiperparámetros sobre un conjunto fijo de algoritmos definidos en el código. La evolución propuesta consiste en ampliar esta lógica para que la exploración no solo ocurra dentro del espacio de hiperparámetros, sino también a nivel de selección de modelo.

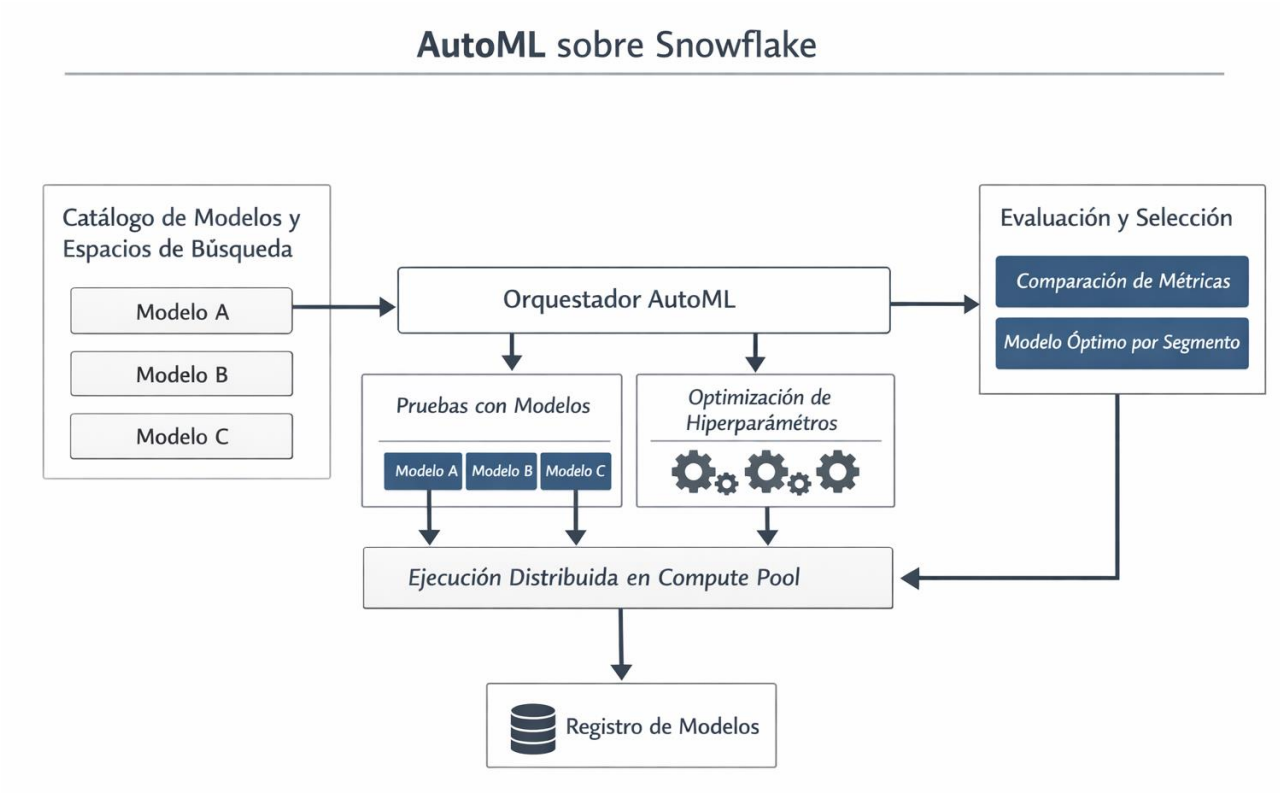
Conceptualmente, esto se implementaría mediante:

- Un diccionario o tabla de configuración que defina múltiples algoritmos candidatos (por ejemplo, XGBoost, LightGBM, modelos lineales, etc.).
- Para cada algoritmo, un espacio de búsqueda de hiperparámetros estructurado.
- Una capa de orquestación que itere sobre combinaciones de (modelo, espacio de búsqueda, segmento).
- Registro unificado de métricas en Model Registry para comparar desempeño entre algoritmos y configuraciones.

En este esquema, cada ejecución evaluaría múltiples combinaciones modelo-configuración, distribuidas sobre el compute pool mediante HPO. Posteriormente, se seleccionaría automáticamente el mejor modelo por segmento según la métrica objetivo definida.

Este enfoque no forma parte del alcance actual de la POC, pero puede implementarse como una capa adicional de orquestación sobre HPO y MMT, manteniéndose completamente dentro del ecosistema Snowflake. De esta manera, se simula un comportamiento AutoML sin depender de una solución externa propietaria.

A continuación, la representación gráfica de la solución propuesta:

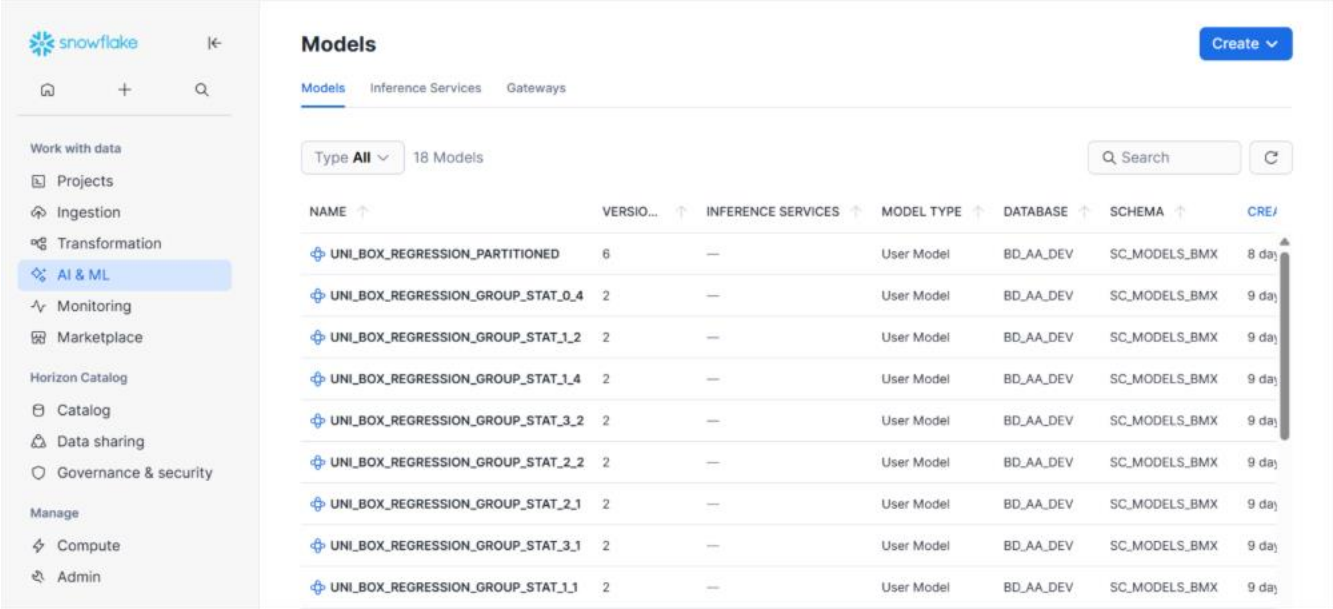


9. Model Registry

▶ POC

9.1 Función y Estructura

El Snowflake Model Registry es un repositorio integrado que centraliza el gobierno del ciclo de vida de los modelos directamente dentro de Snowflake, reemplazando MLflow como componente externo. Permite gestionar múltiples versiones, comparar performance entre ellas y controlar qué versión está activa en cada ambiente.



9.2 Artefactos Almacenados por Versión

Cada versión registrada en el Model Registry almacena los siguientes artefactos y metadatos:

Artefacto / Metadato	Descripción
Modelo serializado	Objeto del modelo entrenado (XGBRegressor / LGBMRegressor) serializado en formato compatible con Snowflake ML.
Métricas de performance	RMSE, MAE, WAPE y MAPE por grupo (16 valores de cada métrica) registradas en el momento del entrenamiento.
Hiperparámetros	Configuración óptima encontrada por HPO para cada grupo (n_estimators, max_depth, learning_rate, etc.).
Tags de versión activa	Tags asignados al modelo para identificar qué versión está activa por proyecto o grupo de consumo (ej. act_general, act_grupo1). Controlados vía RBAC por ML_OPS_ROLE.
Autor y timestamp	Usuario que registró el modelo y fecha/hora de registro para trazabilidad y auditoría.
Dependencias	Versiones de librerías utilizadas (XGBoost, LightGBM, Snowflake ML) para reproducibilidad del entorno.

10. Inferencia Batch Particionada

▶ POC

10.1 Funcionamiento

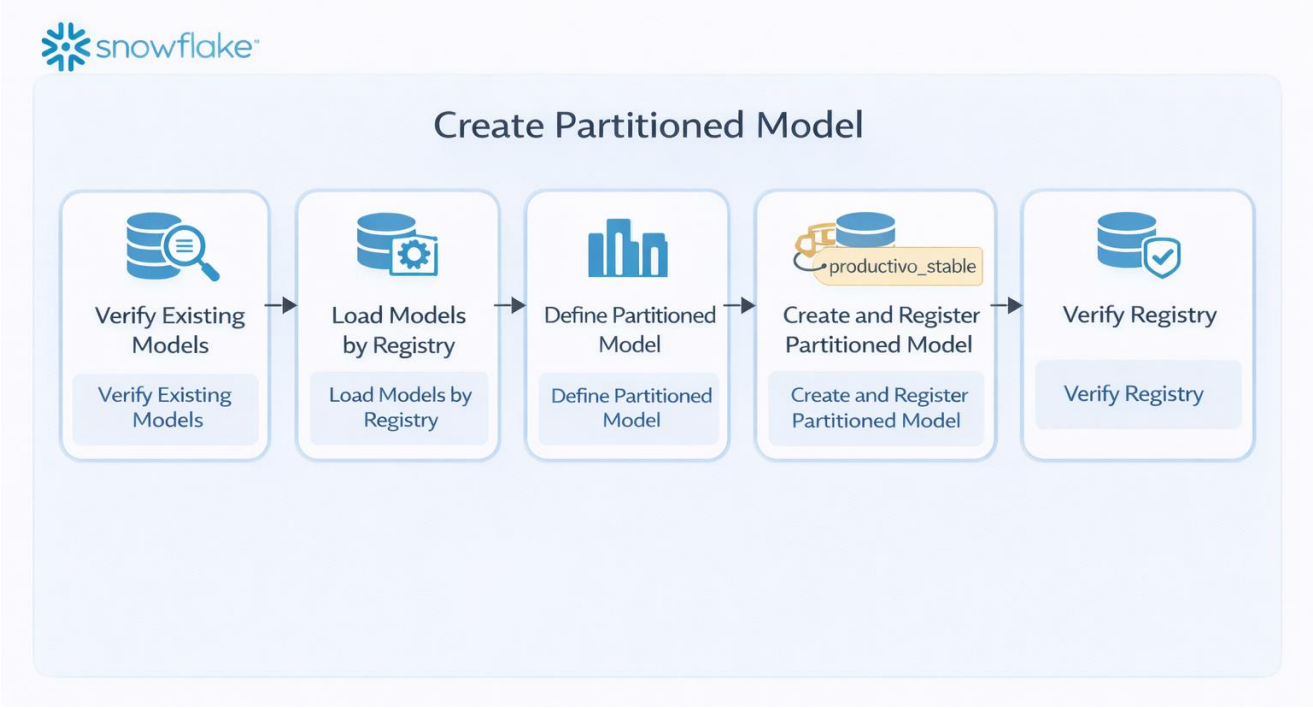
Partitioned Model Inference es el complemento natural de MMT. Una vez registrado el modelo particionado en el Registry, el motor de Snowflake resuelve automáticamente qué sub-modelo corresponde a cada registro en función del valor de la columna STATS_NTILE_GROUP, sin necesidad de lógica condicional adicional ni enrutador externo.

El modelo particionado encapsula los 16 sub-modelos de segmento como una sola unidad registrada en el Model Registry, lo que simplifica su gestión: se versiona, se promueve entre ambientes y se audita como un único objeto, independientemente del número de grupos.

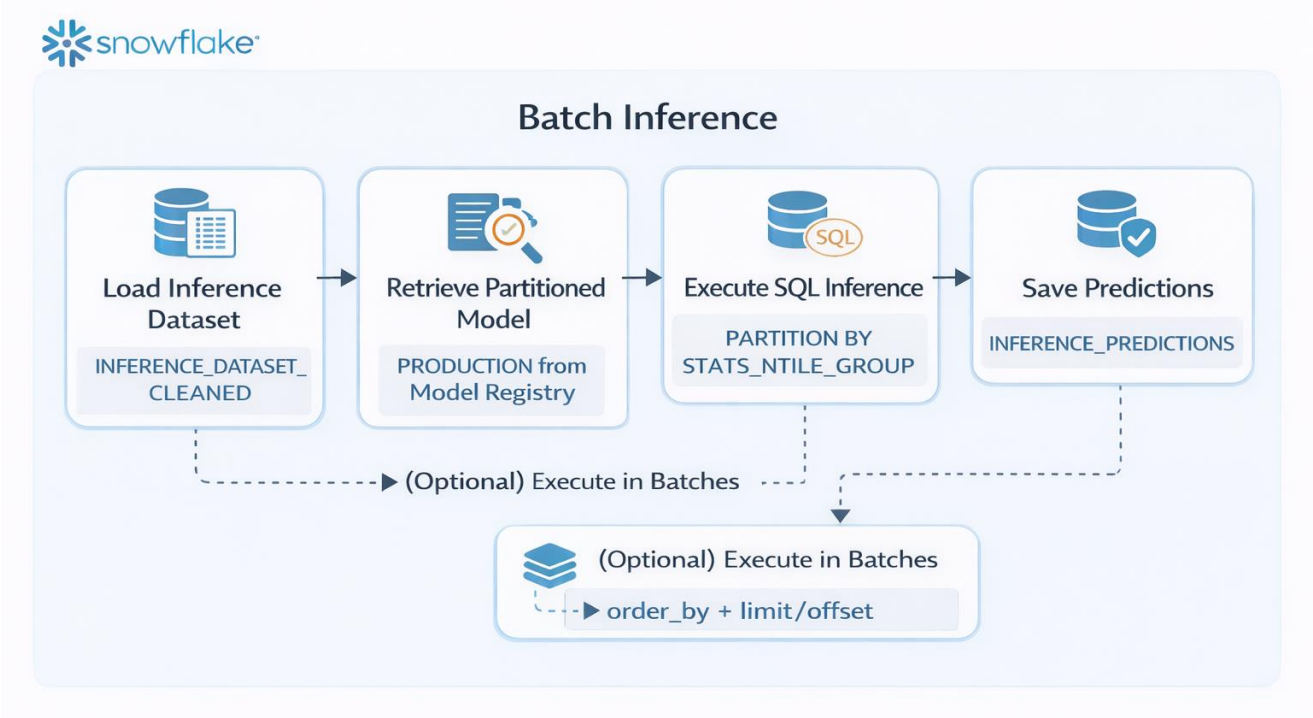
Creación del Modelo Particionado

La construcción del modelo particionado sigue cinco pasos secuenciales:

1. **Verificar modelos existentes:** Se consulta el Model Registry para confirmar que los 16 sub-modelos entrenados por MMT están disponibles y correctamente registrados, con sus versiones y métricas.
2. **Cargar modelos por Registry:** Se recuperan las referencias a cada sub-modelo desde el Registry, resolviendo nombre, versión y columna de partición asociada (valor de STATS_NTILE_GROUP). Este paso construye el mapeo entre cada segmento y su modelo correspondiente.
3. **Definir el Modelo Particionado:** Se define el wrapper del modelo particionado, especificando la columna de partición (STATS_NTILE_GROUP) y la función de predicción que será invocada para cada segmento.
4. **Crear y Registrar el Modelo Particionado:** Se crea el objeto particionado en Snowflake y se registra en el Model Registry como una entidad unificada (UNI_BOX_REGRESSION_PARTITIONED). En este paso se asigna la versión y el Tag de versión activa correspondiente (por ejemplo, `productivo_stable`), que será el punto de referencia para los procesos de inferencia y para la promoción entre ambientes.
5. **Verificar Registry:** Se valida que el modelo particionado quedó correctamente registrado, que la columna de partición está bien configurada, y que es invocable desde SQL nativo antes de proceder a la fase de inferencia.



10.2 Flujo de Ejecución



El proceso de inferencia batch sigue este flujo:

1. Se carga el dataset de inferencia desde la tabla fuente (INFERENCE_DATASET_CLEANED). Para la etapa de la POC se usa esta tabla, pero en Prod Cercano se usará el Feature Store para generar las tablas de entrenamiento e inferencia.
2. Se recupera la referencia al modelo particionado (por ejemplo, con el tag 'production_stable') desde el Model Registry.

3. Se ejecuta la inferencia mediante sintaxis SQL nativa con la cláusula TABLE(MODEL(...)!PREDICT(...) OVER (PARTITION BY STATS_NTILE_GROUP)).
4. Los resultados se insertan en la tabla INFERENCE_PREDICTIONS con los campos: CUSTOMER_ID, STATS_NTILE_GROUP, WEEK, BRAND_PRES_RET, PROD_KEY, PREDICTED_UNI_BOX_WEEK, MODEL_VERSION y PREDICTION_TIMESTAMP.
5. Opcionalmente, si el volumen lo requiere, la inferencia puede ejecutarse en batches (BATCH_SIZE configurable) usando Snowpark order_by + limit/offset.

```
-- Sintaxis SQL nativa de inferencia particionada
INSERT INTO INFERENCE_PREDICTIONS (...)
SELECT p.CUSTOMER_ID, p.WEEK, p.predicted_uni_box_week, ...
FROM INFERENCE_INPUT t,
TABLE(
  MODEL(BD_AA_DEV.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED, TAG =>
'production_stable')
  !PREDICT(t.CUSTOMER_ID, t.STATS_NTILE_GROUP, t.WEEK, ...)
  OVER (PARTITION BY t.STATS_NTILE_GROUP)
) p
```

Deployment en producción

La estrategia de deployment está definida: Tags para gestión de versiones activas por proyecto, y Export/Import de artefactos para la promoción entre ambientes DEV → QA → PROD. Ver sección 12 para el detalle completo.

11. ML Observability — Monitoreo de Datos y Modelos

11.1 Estrategia de Monitoreo

▶ POC

El monitoreo de ML en la POC se implementa mediante un enfoque completamente custom en Python/Snowpark, sin uso del módulo nativo de ML Observability de Snowflake. Esta decisión responde a que el módulo nativo no cubre los requerimientos de granularidad por segmento (STATS_NTILE_GROUP) que el modelo requiere. La estrategia se basa en tres pilares:

Tipo de Drift	Qué detecta	Métricas utilizadas
Data Drift	Cambios en la distribución de las features de entrada entre el set de entrenamiento y los datos de inferencia.	Jensen-Shannon Distance (por feature, por segmento), Population Stability Index (PSI por segmento).
Performance Drift	Degradación de las métricas del modelo respecto al baseline del set de entrenamiento.	WAPE, RMSE, MAE, F1 Binary. Calculados por segmento y a nivel agregado ponderado.
Prediction Drift	Cambios en la distribución de predicciones del modelo.	Jensen-Shannon Distance sobre la distribución de PREDICTED_UNI_BOX_WEEK, por segmento.

Las métricas adicionales mencionadas en versiones anteriores del documento (KS, Wasserstein, Chi-square, Cramér's V) **no están implementadas en la POC**. Su incorporación queda como punto de evolución para producción cercana, según lo que Arca Continental priorice.

11.2 Visualización por Segmento y Agregada

El módulo de Performance Drift produce resultados en dos niveles:

- **Por segmento (STATS_NTILE_GROUP):** métricas calculadas individualmente para cada uno de los 16 grupos, permitiendo identificar qué segmento específico está degradándose.
- **Agregado ponderado (full_model):** las métricas se agregan sobre todos los grupos mediante suma ponderada, no promedio simple. Esto evita que los grupos de baja rotación distorsionen la visión ejecutiva del comportamiento del modelo.

El histórico contra el que se compara el Performance Drift es el **baseline calculado a partir de las predicciones del set de entrenamiento** (TRAIN_PREDICTIONS). El drift se expresa como variación proporcional relativa al baseline: umbral de Warning en +20% y Critical en +50% para métricas de error (WAPE, RMSE, MAE); Warning en -15% y Critical en -30% para F1, donde una caída es lo que indica degradación.

11.3 Configuración del Monitoreo

▶ POC

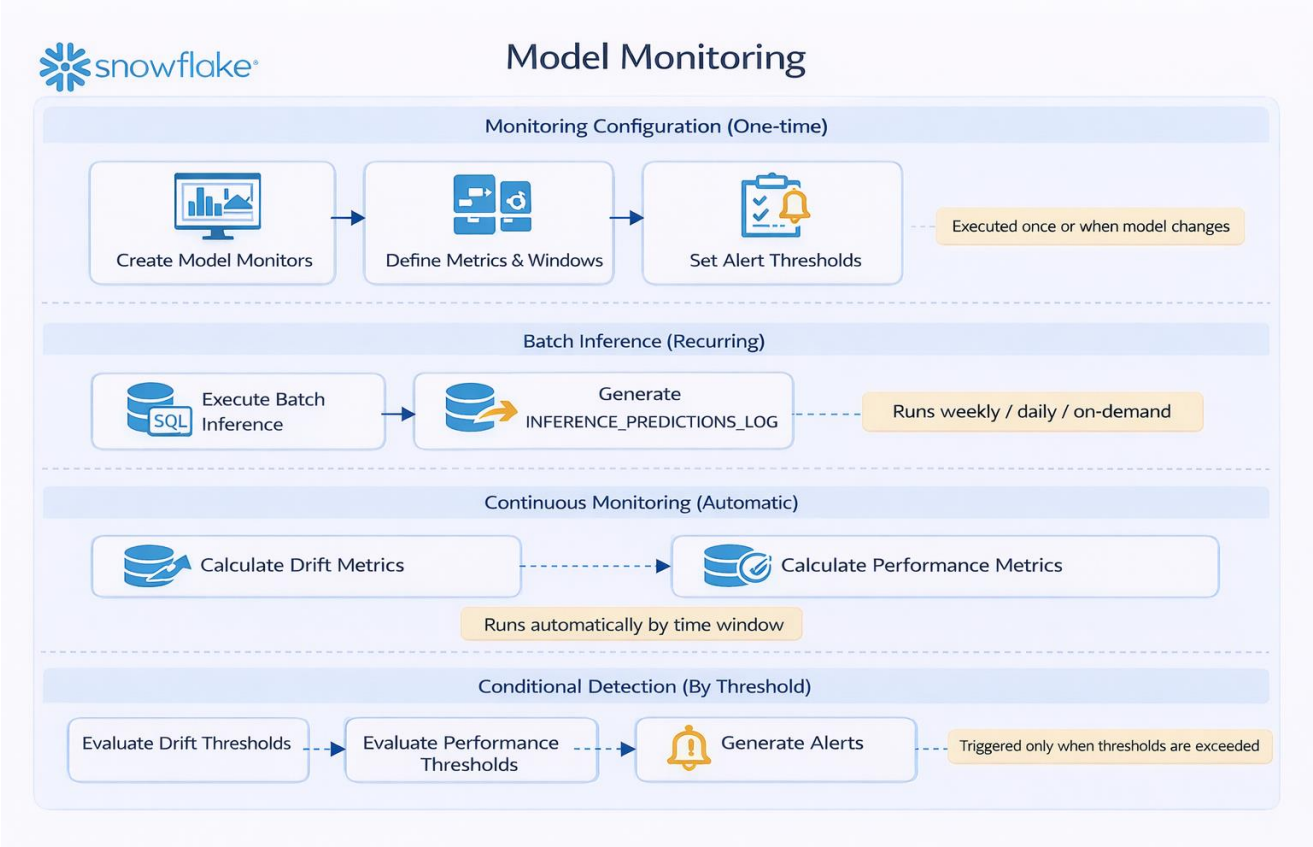
En la POC, el monitoreo se implementa programáticamente mediante Python y SQL con Snowpark, aprovechando el *micropartitioning* nativo de Snowflake para el procesamiento eficiente de los logs. El flujo es:

1. **Configuración inicial (una vez por versión de modelo):** se crean las tablas físicas de landing para histogramas y métricas de drift. Esta configuración no requiere repetirse salvo que se registre una nueva versión de modelo.

2. **Cálculo del baseline (una vez por versión de modelo):** se calculan los histogramas de referencia de features y predicciones, y las métricas de performance base, a partir del set de entrenamiento en DEV. Los resultados se almacenan en tablas físicas (`DA_DATA_DRIFT_HISTOGRAMS_BASELINE`, `DA_PREDICTION_DRIFT_HISTOGRAMS_BASELINE`, `DA_PERFORMANCE_BASELINE`) y se copian junto al modelo al promoverlo a QA y PROD.
3. **Cálculo incremental (por cada nueva semana de inferencia):** el proceso detecta automáticamente qué combinaciones (WEEK, MODEL_VERSION) no han sido procesadas aún y calcula únicamente los nuevos registros. Esto lo hace idempotente: puede ejecutarse varias veces sin duplicar datos. Los resultados se materializan en `DA_DATA_DRIFT`, `DA_PREDICTION_DRIFT` y `DA_PERFORMANCE_DRIFT`.
4. **Visualización:** los dashboards de Snowsight se construyen sobre estas tablas físicas. La actualización de las métricas depende de la frecuencia con que se ejecute el notebook de monitoreo, que puede ser bajo demanda o periódico según lo que Arca Continental defina.

Tablas de soporte y su relación con el Model Registry

El baseline de drift es una tabla física (`DA_DATA_DRIFT_HISTOGRAMS_BASELINE`) almacenada en el schema de monitoreo, no un artefacto adjunto al objeto modelo en el Registry. Esto implica que al copiar el modelo entre ambientes (DEV → QA → PROD), las tablas de baseline **deben copiarse explícitamente** mediante CLONE o INSERT SELECT. No se copian automáticamente con el `CREATE MODEL`. El procedimiento de copia está documentado en la sección 12.3.



Importancia de micropartitioning

El micropartitioning es el mecanismo de almacenamiento físico de Snowflake. Cuando se carga datos en una tabla, Snowflake los divide automáticamente en particiones pequeñas e inmutables de entre 50 y 500

MB comprimidos, organizadas por orden de inserción y con metadatos de rango de valores por columna almacenados por separado.

Esto es relevante para el monitoreo porque cuando el proceso Python/SQL consulta los logs de inferencia filtrando por ventana temporal, grupo o versión de modelo, Snowflake usa esos metadatos para descartar automáticamente las particiones que no contienen datos relevantes — sin necesidad de índices ni particionamiento manual. El resultado es que los cálculos de drift y performance sobre subconjuntos grandes de logs son significativamente más eficientes sin requerir configuración adicional.

12. Estrategia de Deployment y Promoción de Modelos

Esta sección describe la estrategia completa de deployment: separación de ambientes, gestión de múltiples versiones activas mediante Tags, y promoción de modelos entre ambientes mediante comandos SQL nativos de Snowflake (CREATE MODEL ... WITH VERSION ... FROM MODEL ...) junto con la copia de tablas de soporte para ML Observability.

12.1 Separación por Ambientes

La estrategia define tres ambientes separados por base de datos en Snowflake, con roles de acceso diferenciados:

Ambiente	Base de Datos	Propósito
DEV	ML_DEV_DB	Desarrollo, experimentación, entrenamiento y validación inicial. Acceso a ML_DEV_ROLE.
QA / Staging	ML_QA_DB	Validación funcional y de calidad antes de producción. Acceso a ML_OPS_ROLE.
PROD	ML_PRD_DB	Ambiente productivo. Solo ML_OPS_ROLE puede realizar promociones. Los consumidores operan con ML_CONSUMER_ROLE.

12.2 Gestión de Versiones Activas mediante Tags

Arca Continental opera con un escenario donde un mismo modelo puede tener múltiples versiones activas simultáneamente en producción, asociadas a distintos proyectos o grupos de consumo. La solución adoptada es el uso de Tags en el Snowflake Model Registry.

Un Tag es una etiqueta con valor variable asignada directamente al objeto modelo, manipulable mediante SQL y con permisos controlables vía RBAC. Cada proyecto define su propio Tag, permitiendo que distintos consumidores apunten a versiones diferentes del mismo modelo sin conflictos y sin modificar el código de inferencia:

```
-- Asignar versión activa por proyecto

ALTER MODEL UNI_BOX_REGRESSION_PARTITIONED SET TAG act_general  = 'v20260103';
ALTER MODEL UNI_BOX_REGRESSION_PARTITIONED SET TAG act_grupo1   = 'v20260103';
ALTER MODEL UNI_BOX_REGRESSION_PARTITIONED SET TAG act_grupo2   = 'v20251205';


-- Consultar versión activa en el proceso de inferencia SET version = (SELECT
SYSTEM$GET_TAG('act_grupo1', 'UNI_BOX_REGRESSION_PARTITIONED', 'MODULE'));
```

Ventajas: control de acceso granular (solo ML_OPS_ROLE modifica tags de producción), trazabilidad completa en QUERY_HISTORY, y sin cambios de código en procesos consumidores al actualizar una versión. El límite de 50 tags por modelo deja margen para los 16 grupos actuales con capacidad de crecimiento.

12.3 Flujo de Promoción DEV → QA → PROD

El modelo NO se reentrena en producción. Se entrena UNA VEZ en DEV, se valida en QA y se promueve a PROD mediante sentencias SQL nativas de Snowflake. La promoción involucra dos elementos: el modelo particionado y las tablas de soporte requeridas por ML Observability.

Copia del Modelo Particionado entre Ambientes

La promoción del modelo se realiza mediante CREATE MODEL ... WITH VERSION ... FROM MODEL ... VERSION ..., especificando los nombres completos de base de datos, schema, modelo y versión en origen y destino. Solo es necesario copiar el modelo particionado (UNI_BOX_REGRESSION_PARTITIONED): este contiene internamente los 16 modelos de segmento, por lo que no es necesario copiarlos individualmente.

Los modelos de segmento individuales solo necesitarían copiarse a otro ambiente si fuera necesario acceder a elementos de su metadata específica, ya que esta metadata granular no se incluye automáticamente en la copia del modelo particionado.

```
-- Copiar modelo particionado de DEV a QA

CREATE MODEL ML_QA_DB.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED WITH VERSION v20260103
FROM MODEL ML_DEV_DB.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED VERSION v20260103;

-- Copiar de QA a PROD (mismo patrón)

CREATE MODEL ML_PRD_DB.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED WITH VERSION
v20260103 FROM MODEL ML_QA_DB.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED VERSION
v20260103;

-- Actualizar tag de versión activa en el ambiente destino

ALTER MODEL ML_PRD_DB.SC_MODELS_BMX.UNI_BOX_REGRESSION_PARTITIONED SET TAG act_general =
'v20260103';
```

Copia de Tablas de Soporte para ML Observability

Además del modelo, las tablas de soporte del monitoreo deben copiarse explícitamente entre ambientes al promover una nueva versión. Estas tablas son tablas físicas estándar de Snowflake almacenadas en el schema de monitoreo — **no son artefactos adjuntos al objeto modelo en el Registry**, y por tanto no se copian automáticamente con el `CREATE MODEL`.

Esta distinción es importante: el Snowflake Model Registry soporta adjuntar objetos JSON arbitrarios como metadata de una versión (lo que permite registrar coeficientes custom, configuraciones o cualquier valor serializable), pero las tablas de datos — por su volumen y naturaleza relacional — se gestionan fuera del objeto modelo como tablas físicas independientes. Copiarlas es un paso explícito y deliberado del flujo de promoción, no un gap del proceso.

Las dos tablas que deben copiarse son:

- **`DA_DATA_DRIFT_HISTOGRAMS_BASELINE`** — contiene los histogramas de referencia de features y predicciones calculados a partir del set de entrenamiento. Sin esta tabla en el ambiente destino, el proceso de monitoreo no puede calcular Jensen-Shannon Distance ni PSI, ya que no tiene distribución de referencia contra la cual comparar.

- **`DA_PERFORMANCE_BASELINE`** — contiene las métricas de performance base (WAPE, RMSE, MAE, F1 por segmento y agregado) calculadas sobre las predicciones del set de entrenamiento. Sin esta tabla, el cálculo de Performance Drift no puede determinar la variación proporcional respecto al estado original del modelo.

La copia se realiza una sola vez por versión de modelo, como parte del paso de promoción:

```
-- Copiar baseline de histogramas de DEV a QA CREATE TABLE
ML_QA_DB.SC_MODELS_BMX.DA_DATA_DRIFT_HISTOGRAMS_BASELINE CLONE
ML_DEV_DB.SC_MODELS_BMX.DA_DATA_DRIFT_HISTOGRAMS_BASELINE;

-- Copiar baseline de performance de DEV a QA INSERT INTO
ML_QA_DB.SC_MODELS_BMX.DA_PERFORMANCE_BASELINE SELECT * FROM
ML_DEV_DB.SC_MODELS_BMX.DA_PERFORMANCE_BASELINE WHERE MODEL_VERSION = 'v20260103';
```

Validación y Mecanismo de Promoción

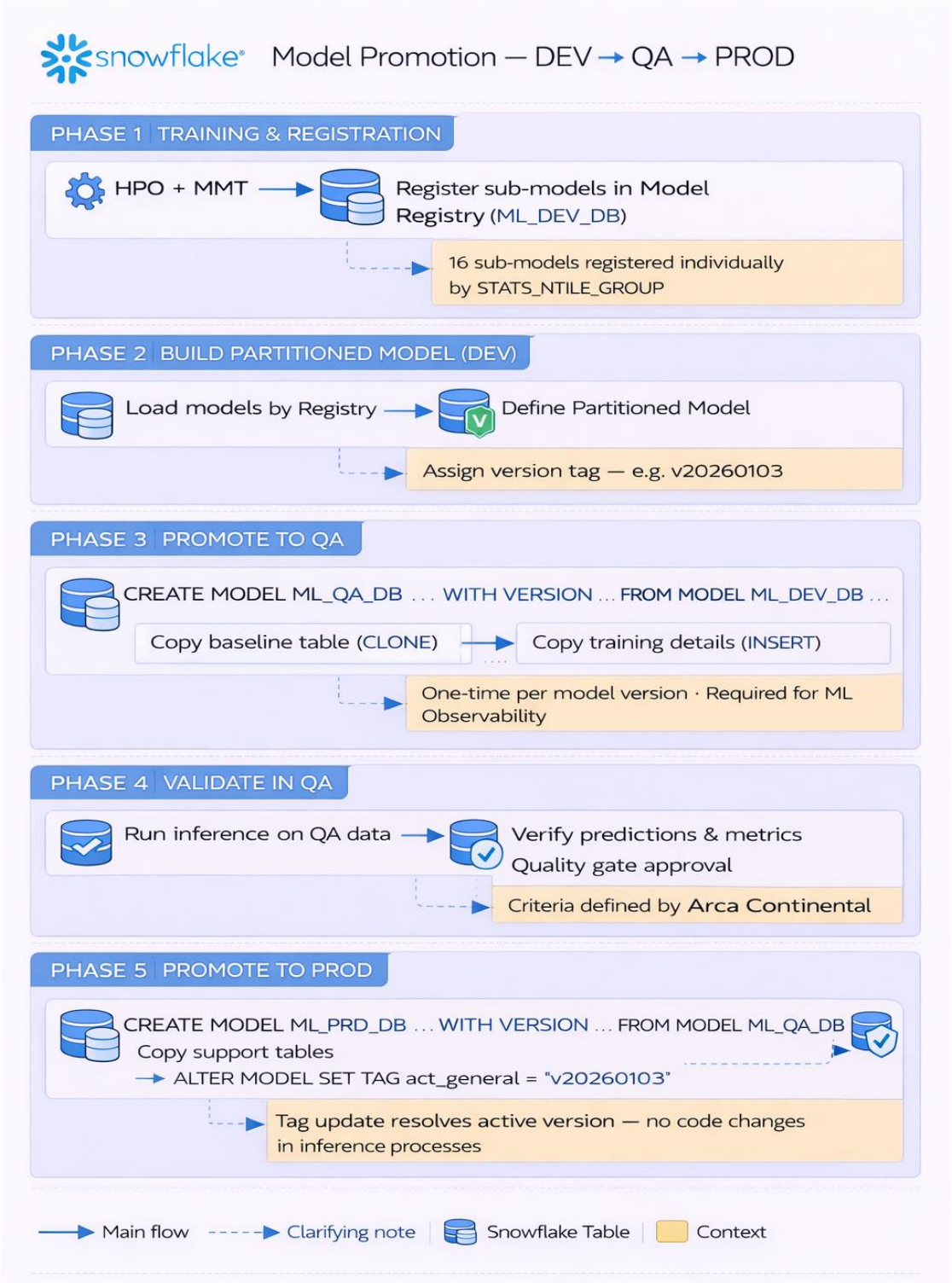
La validación necesaria para aprobar el cambio de ambiente depende de lo que Arca Continental defina como criterios de aceptación. Igualmente, el mecanismo para efectuar la promoción es flexible:

- Ejecución manual: ML_OPS_ROLE ejecuta las sentencias SQL de copia directamente, adecuado para la POC y producción cercana donde las promociones son poco frecuentes.
- Script parametrizado: las sentencias se encapsulan en un script ligero con parámetros (modelo, versión, ambiente origen, ambiente destino), ejecutable bajo demanda por ML_OPS_ROLE. • Integración en DAG: si existe un orquestador (Dagster u otro), las operaciones de copia pueden incorporarse como un nodo del pipeline, ejecutándose automáticamente tras la aprobación del gate de calidad.

En todos los casos, el flujo de promoción completo es el siguiente:

1. Entrenamiento en DEV: HPO + MMT se ejecutan. Los modelos se registran en ML_DEV_DB.SC_MODELS con versión, métricas y metadatos.
2. Validación en DEV: el equipo ML revisa métricas por grupo (RMSE, MAE, WAPE) como gate de calidad.
3. Copia a QA: CREATE MODEL de DEV a QA + copia de tablas de baseline y detalles de entrenamiento.
4. Pruebas en QA: ejecución de inferencia funcional, verificación de consistencia de predicciones y monitoreo.
5. Aprobación de QA: gate de calidad final antes de promover a PROD.
6. Copia a PROD: CREATE MODEL de QA a PROD + copia de tablas de soporte.
7. Actualización de Tag en PROD: ML_OPS_ROLE asigna el Tag del proyecto a la nueva versión. Los procesos de inferencia resuelven la versión activa leyendo el Tag, sin cambios de código.

A continuación, se muestra una ilustración del flujo:



12.4 Control de Acceso RBAC

▶ PROD. CERCANO

El gobierno del ciclo de vida se implementa mediante cuatro roles diferenciados:

Rol	Permisos
ML_DEV_ROLE	Lectura/escritura en schemas DEV. Registro de modelos en Model Registry de DEV. Sin acceso a QA ni PROD.
ML_OPS_ROLE	Permisos completos para Export/Import de modelos y actualización de Tags en todos los ambientes. OWNERSHIP en schemas de producción. Único rol autorizado para modificar versiones activas en PROD.
ML_CONSUMER_ROLE	Solo lectura y ejecución de modelos en producción (SELECT/USAGE sobre funciones de inferencia). Sin permisos de modificación de modelos ni Tags.
ML_MONITOR_ROLE	Lectura de logs de inferencia, métricas y tablas de monitoreo. Sin permisos de modificación sobre modelos o schemas de entrenamiento.

13. Orquestación del Pipeline

13.1 Orquestación Manual

En la POC, el pipeline se ejecuta de forma manual mediante notebooks orquestados paso a paso. La secuencia es la siguiente:

Paso	Descripción	Frecuencia	Duración estimada
1. Validación y limpieza de datos	Verificación de estructura, calidad y compatibilidad entre los datasets de entrenamiento e inferencia.	Una vez por ciclo, antes de entrenar	—
2. Construcción del Feature Store	Cálculo de features mediante Window Functions SQL (lags, rolling aggregations) y materialización en tablas físicas estáticas.	Una vez por ciclo	—
3. HPO Bayesian Search	Búsqueda de hiperparámetros óptimos por grupo (15 trials por grupo, cluster de 5 nodos M).	Puntual, bajo demanda ante degradación o cambio significativo de datos	**~1h**
4. MMT	Entrenamiento paralelo de los 16 sub-modelos con hiperparámetros validados (cluster de 5 nodos M).	Una vez por ciclo de reentrenamiento	**~7 min**
5. Construcción del Modelo Particionado	Verificar modelos en Registry, cargar referencias, definir particionado, crear y registrar modelo particionado con Tag de versión activa, verificar Registry.	Una vez por ciclo de reentrenamiento, tras MMT	—
6. Promoción entre ambientes	Copia del modelo particionado (CREATE MODEL), copia de tablas de baseline y detalles de entrenamiento para ML Observability, actualización del Tag en el ambiente destino.	Una vez por versión de modelo	—
7. Inferencia batch	Ejecución de predicciones con sintaxis SQL nativa sobre el dataset de inferencia, consumiendo el modelo particionado registrado.	Recurrente (frecuencia semanal, mensual, etc)	—
8. Configuración de ML Observability	Creación de dashboards de drift en Snowsight (Data Drift, Performance Drift, Prediction Drift). La actualización de métricas es automática según la ventana configurada; solo requiere intervención ante cambio de umbrales o nueva versión de modelo.	Configuración: una vez por versión de modelo. Actualización de métricas: automática (horaria, diaria u on-demand según configuración)	—

13.2 Orquestación Automatizada

▶ **PROD. CERCANO**

En producción cercano, el pipeline se automatiza mediante Snowflake Tasks, ML Jobs o un orquestador externo (Dagster). El DAG de dependencias propuesto es:

1. **Task/Job:** Validación y limpieza de datos entrantes.
2. **Task/Job dependiente:** Actualización del Feature Store — pipeline batch incremental completo: selección de PENDING, cálculo de features, INSERT en HIST, DROP/RECREATE de LATEST, cierre de FEATURE_BATCH_RUNS.
3. **Task/Job condicional:** Re-entrenamiento (MMT) si se detecta drift significativo o se cumple la periodicidad definida. Incluye construcción del modelo particionado y registro en Model Registry.
4. **Task/Job de promoción:** si el modelo supera el gate de calidad, copia automática o semi-automática a QA/PROD via CREATE MODEL + copia de tablas de soporte, y actualización de Tags.
5. **Task/Job programado:** Inferencia batch sobre FEATURES_UNI_BOX_LATEST.
6. **Task/Job de monitoreo:** Generación de estadísticas de drift y performance, y disparo de alertas si se superan umbrales definidos.

14. Comparativa: POC vs. Producción

La siguiente tabla diferencia tres horizontes: la POC ya implementada, la producción cercana (mejoras inmediatas sin grandes cambios de arquitectura) y la producción lejana (evolución completa del sistema a mediano/largo plazo):

Aspecto	POC (actual)	Prod. Cercano	Prod. Lejano
Datos	Históricos estáticos curados por Arca Continental	Mismos datos con proceso de validación y limpieza documentado	Actualización periódica automatizada desde fuentes origen
Inferencia	Batch manual sobre dataset de inferencia curado	Batch periódico con trigger definido, consumiendo FEATURES_UNI_BOX_LATEST	Batch + Real-time (SPCS con endpoints HTTPS)
Feature Store	Tablas físicas estáticas, cálculo manual de lags mediante Window Functions SQL	Batch incremental: HIST (append-only) + LATEST (DROP/RECREATE por lote), trazabilidad por BATCH_ID	Tasks / ML Jobs nativos o FeatureViews con governance integrado
HPO	Manual, puntual, una sola ejecución	Bajo demanda ante degradación de métricas detectada	Trigger automático por drift
MMT — Reentrenamiento	Manual, una vez por ciclo de experimentación	Periódico (frecuencia por definir con Arca Continental)	Automático con aprobación basada en umbrales de métricas
Modelo Particionado	Construcción manual en 5 pasos: verificar modelos, cargar por Registry, definir particionado, crear y registrar, verificar Registry	Mismo proceso, incorporado en script parametrizado ejecutado por ML_OPS_ROLE	Construcción automatizada como parte del pipeline de reentrenamiento
Orquestación	Ejecución manual de notebooks paso a paso	Orquestador externo (Dagster) o scripts parametrizados controlados por ML_OPS	Snowflake Tasks / ML Jobs con DAG de dependencias y logging nativo
Deployment	CREATE MODEL entre ambientes + Tags por proyecto, ejecución manual con copia de tablas de soporte para Observability	Script parametrizado o nodo en DAG; copia de baseline y detalles de entrenamiento una vez por versión	Pipeline CI/CD automatizado con gates de calidad por etapa
Escalamiento	Warehouse X-Small/Small, Compute pool de 5 nodos M	Warehouse Medium/Large Optimizado; ajuste manual según carga. Compute pool de 5 nodos M o 2 nodos L de acuerdo a la carga. Sugerencia basada en análisis de Snowflake.	Warehouse Large/X-Large con multi-cluster y auto-scaling dinámico
Versionado multi-proyecto	Tags por proyecto en Model Registry (act_general, act_grupo1, etc.)	Tags + copia nativa CREATE MODEL con auditoría en QUERY_HISTORY	Gobierno centralizado de versiones con trazabilidad completa

Monitoreo	Monitoreo en Python/SQL con Snowpark. Baseline en tabla física generado en DEV. Dashboards en Snowsight con umbrales Warning/Critical definidos.	Umbrales refinados con base en la POC. Baseline y detalles de entrenamiento copiados junto al modelo en cada promoción.	Dashboards avanzados, alertas proactivas y auto-reentrenamiento por drift.
Selección de modelos	Algoritmo fijo por segmento (definido en código, asignado por HPO)	Selección validada manualmente por el equipo ML. Opcionalmente tambien se puede implementar la propuesta AutoML en una 1era versión.	Propuesta AutoML: selección automática del mejor modelo por segmento

15. Consideraciones y Responsabilidades

15.1 Responsabilidades de Arca Continental

1. Proveer accesos a Snowflake con los permisos necesarios para trabajar en el ambiente de desarrollo.
2. Participar en la transferencia de conocimiento sobre la funcionalidad actual del modelo en Databricks.
3. Entregar el dataset de entrenamiento y el de inferencia de forma curada y lista para usarse como fuente de la POC.
4. Acompañar en la fase de certificación de datos para la aprobación de la migración de los modelos creados.
5. Definir y validar los umbrales de aceptación de métricas (RMSE, WAPE) que determinarán la aprobación del modelo en cada ambiente.
6. Confirmar la estrategia de versionado multi-proyecto (aliases vs. Tags) y la frecuencia de reentrenamiento.

15.2 Responsabilidades de Seidor Analytics

1. Desarrollar la migración técnica del modelo de Pronóstico de Ventas Semanales desde Databricks hacia Snowflake.
2. Implementar las mejores prácticas con las herramientas nativas de Snowflake para maximizar el aprovechamiento de la plataforma.
3. Entregar los modelos entrenados y las predicciones versionadas y certificadas para su aprobación.
4. Entregar documentación de desarrollo, certificación de datos y playbook de componentes desarrollados.
5. Definir y documentar formalmente la estrategia de deployment en producción una vez alineada con Arca Continental.